Unisoc Confidential For longcheer

Android 音频简介及常见问题分析

文档版本	V1.0
发布日期	2022-11-30

紫光展锐（上海）科技有限公司

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、UNISOC、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

Bluetooth®文字商标和徽标为蓝牙技术联盟的注册商标，紫光展锐（上海）科技有限公司对此类商标的任何使用均已获得许可。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

Unisoc Confidential For longcheer

紫光展锐（上海）科技有限公司

前 言

概述

本文档主要描述了音频基础知识、Android 设备的音频架构、常用音频调试方法和常见音频问题汇总。

读者对象

本文档主要适用于使用展锐平台进行 Android 系统音频设计、调测相关的工程技术人员。

缩略语

缩略语	英文全名	中文解释
AEC	Acoustic Echo Canceler	回声消除器
AGC	Automatic Gain Control	自动增益控制
ANS	Automatic Noise Suppression	自动噪音抑制
AP	Application Processor	应用处理器
Codec	Coder-Decoder	编解码器
CP	Communication Processor	通信处理器
CPT	Call Progress Tone	呼叫过程音
CS	Circuited Switched	电路交换技术
DAI	Digital Audio Interface	数字音频接口
DTMF	Dual Tone Multi Frequency	双音多频音
DTS	Device Tree Source File	设备树源文件
GPIO	General Purpose Input Output	通用输入输出口
HAL	Hardware Abstract Layer	硬件抽象层
PA	Power Amplifier	功率放大器
PCM	Pulse Code Modulation	脉冲编码调制
PS	Packet Switched	分组交换技术
VAD	Voice Activity Detection	语音活动检测

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.0	2022-11-30	第一次正式发布

Unisoc Confidential For longcheer

关键字

Android、音频、编解码

目 录

1 音频基础知识 1
1.1 采样与采样率 1
1.2 采样位数 2
1.3 声道数 3
1.4 音频的采集和播放 3
1.5 数字音频接口 DAI 4
1.6 音频前后处理 4
1.7 音频编解码 5
1.8 语音通信中的 Tone 5
1.9 响度/音调/音色 7
1.10 VoLTE/VoWiFi/VoIP 8
1.10.1 VoLTE 8
1.10.2 VoWiFi 9
1.10.3 VoIP 10
1.10.4 VoLTE/VoWiFi/VoIP 差异比较 10
1.11 模拟功放/数字功放/Smart PA 10
1.12 耳机和耳机座 11
1.13 音频设备 12
2 Android 设备音频架构 14
2.1 硬件架构 14
2.2 软件架构 15
2.2.1 AP 软件架构 15
2.2.2 DSP 软件架构 17
2.2.3 不同场景音频数据流向 17
2.2.4 音频播放 17
2.2.5 音频录制 19
2.2.6 语音通话 20
3 常用调试方法 22
3.1 YLog 和 Logcat 22
3.2 Dump Audio HAL 22
3.3 Dump Audio DSP 22
3.4 tinyalsa 调试命令 23
3.5 查看声卡与设备 24
3.6 查看 GPIO 状态 25
3.7 lookat 操作寄存器 26

Unisoc Confidential For longcheer

4 常见问题汇总 28
4.1 声卡注册失败 28
4.2 主辅 MIC 录音 30
4.3 双 MIC 录音 32
4.4 外置模拟 PA 32
4.5 外置数字 PA 及第三方 Codec 33
4.6 耳机和耳机座 34
4.7 其它音频问题 35
4.7.1 回音 35
4.7.2 噪音 35
4.7.3 Pop 音 35
4.7.4 没声音 36
4.7.5 音量小 36

Unisoc Confidential For longcheer

图目录

图 1-1 声音的采样和量化 1
图 1-2 人耳可听声波范围 2
图 1-3 模拟信号转换成 4-bit PCM 数字信号的采样和量化 2
图 1-4 5.1 声道家庭影院示意图 3
图 1-5 音频的采集和播放 4
图 1-6 数字音频接口 DAI 4
图 1-7 音频前后处理 5
图 1-8 声音的性质和波形 7
图 1-9 电路交换与分组交换 8
图 1-10 VoLTE 9
图 1-11 VoWiFi 9
图 1-12 Audio PA 在音频通路中的位置 10
图 1-13 模拟 PA 和数字 PA 11
图 1-14 耳机接口 12
图 1-15 耳机座 12
图 1-16 各种各样的音频设备 13
图 2-1 终端设备音频硬件架构 14
图 2-2 AP 侧 Audio 软件架构 15
图 2-3 ALSA 子系统架构 16
图 2-4 Audio DSP 软件架构 17
图 2-5 音频播放软解码 18
图 2-6 音频播放硬解码 19
图 2-7 音频录制 20
图 2-8 语音通话 21
图 4-1 I2S 接口接第三方 Codec 33
图 4-2 按照 ALSA 驱动框架要实现的接口 34

Unisoc Confidential For longcheer

图 4-3 PA 使能脚关掉的波形示例.....36

Unisoc Confidential For longcheer

表目录

表 1-1 位深 2
表 1-2 中国的 CPT Tone 标准 6
表 1-3 DTMF Tone 标准中常用按键音的组成 6
表 1-4 VoLTE/VoWiFi/VoIP 差异比较 10

Unisoc Confidential For longcheer

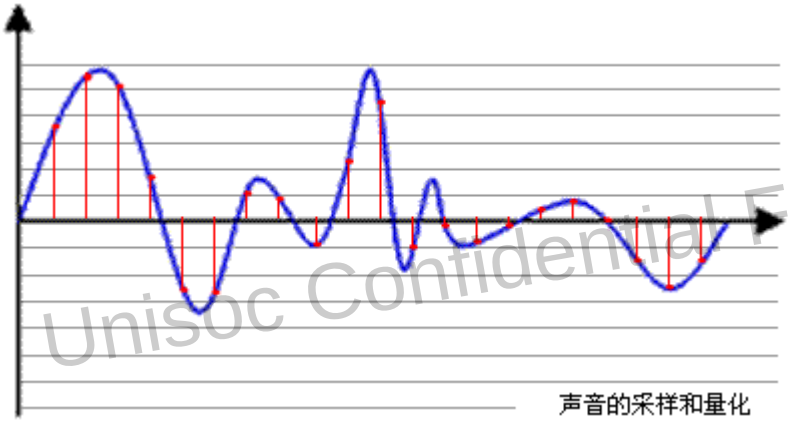
1 音频基础知识

Android 音频子系统是 Android 子系统中较为复杂的系统之一，这不仅仅是从众多分层、模块和代码方面看，还跟复杂的音频设备以及相关音频知识有关。首先需要对这些音频基础知识有一个基本的了解，这样才能对 Android 与 Linux 内核在音频方面如何工作有一个初步的认识。

1.1 采样与采样率

声音是模拟信号，在声音采集时，要把模拟信号变成数字信号，就需要采样，这是 A/D 转换。当要把音频播放出来时，则需要把数字信号转换成模拟信号，这是 D/A 转换。

图1-1 声音的采样和量化



Unisoc Confidential For longcheer

一秒采样的次数称为采样频率，采样频率越高，越接近原始信号，但也加大了运算处理的复杂度。根据**奈奎斯特（Nyquist）采样定理**，要想重建（播放时）原始信号，采样频率必须大于信号中最高频率的两倍。只有采样频率高于原始模拟信号中最高频率的两倍时，才能把数字信号表示的模拟信号准确还原回去。

人能感受到的声音频率范围为 20Hz~20kHz，在现实生活中，音频（Audio）常见的场景有：语音（Voice）和音乐（Music）。

- 语音

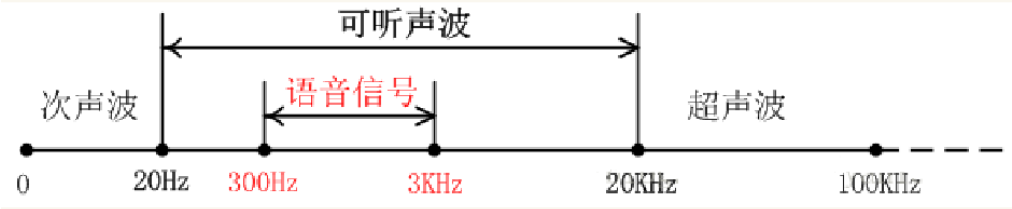
  主要用于沟通通信，如打电话、人机语音交互（如智能音箱），采样率一般较低（窄带 8kHz、宽带 16kHz 和高清语音超宽带 32kHz）。

- 音乐

  一般采样频率为 44.1kHz，更高的可以是 48kHz 和 96kHz，不过一般人用耳听感觉不出差别。

 版权所有 © 紫光展锐（上海）科技有限公司

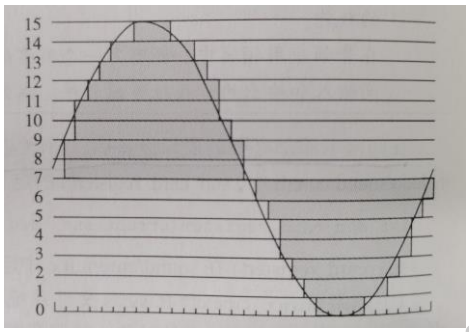
图1-2 人耳可听声波范围



1.2 采样位数

位深（Bit Depth），也叫采样精度，是对模拟信号的振幅进行量化。

图1-3 模拟信号转换成 4-bit PCM 数字信号的采样和量化



Unisoc Confidential For longcheer

采样精度决定了记录声音的动态范围，它以位（Bit）为单位，比如 8 位、16 位、32 位。8 位可以把声波分成 2^8=256 级，16 位可以把同样的声波分成 2^{16}=65536 级。位数越多，对声音的记录就越精细，声音的保真度也就越高。

采样位数的大小影响到声音的质量，数值越大，声音的质量越高，而需要的存储空间也越多；数值越小，声音的质量越低，需要的存储空间也越少。

表1-1 位深

位深（Bit）	品质级别	振幅量化等级数
8	电话	256
16	音频 CD	65,536
24	音频 DVD	16,777,216
32	最佳	4,294,967,296

 版权所有 © 紫光展锐（上海）科技有限公司

1.3 声道数

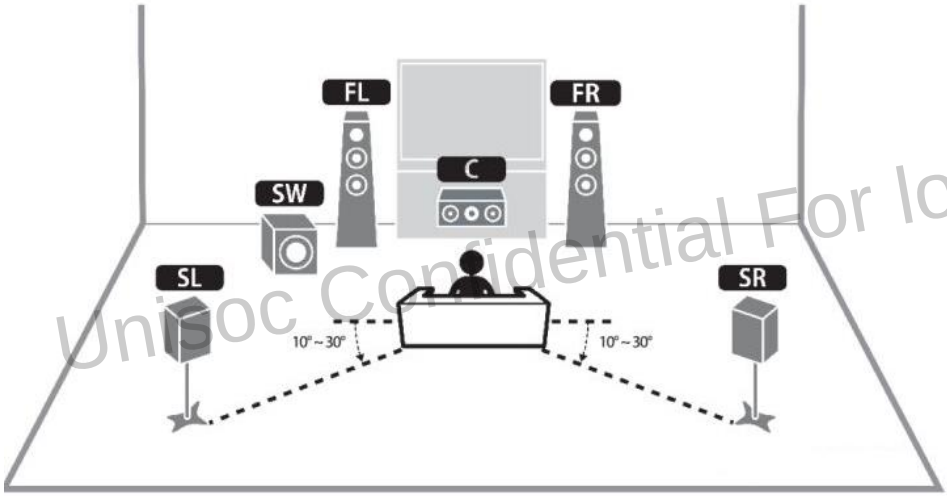
声道是指声音在录制或播放时，在不同空间位置采集或播放的相互独立的音频信号。通俗的说，声道数就是录音时的麦克风数量，或者播放时的喇叭数量。

常见的声道数有单声道（Mono）、双声道（即立体声 Stereo）、5.1 声道、7.1 声道等，这里的.1 声道指的是低音声道。

如图 1-4 所示，是一个 5.1 声道家庭影院示意图，六个扬声器分别是：

- C（Central，中置）
- FL（Front Left，左前置）
- FR（Front Right，右前置）
- SL（Surround Left，左环绕）
- SR（Surround Right，右环绕）
- SW（Subwoofer，低音炮）

图1-4 5.1 声道家庭影院示意图



Unisoc Confidential For longcheer

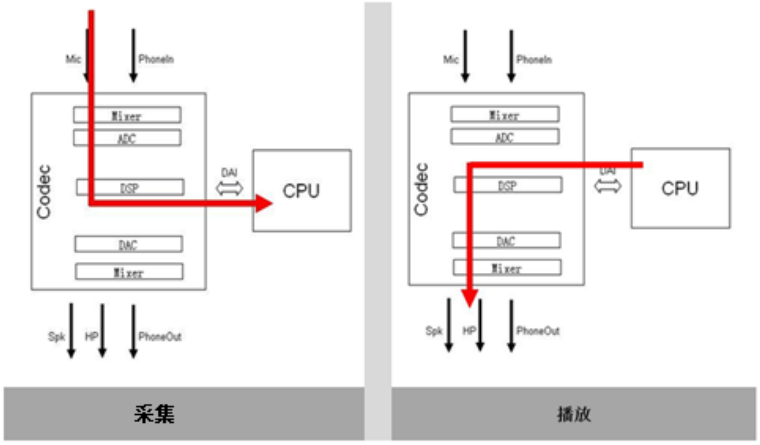
1.4 音频的采集和播放

音频的采集和播放一般采用专门的 Codec（Coder-Decoder，编解码器）芯片。

- 音频采集时，音频模拟信号通过 Codec 内部的 A/D 转换为 PCM（Pulse Code Modulation，脉冲编码调制）数字信号，数字信号通过 DAI（Digital Audio Interface，数字音频接口）传送给 CPU 进行处理。
- 音频播放时，CPU 会把音频数字信号通过 DAI 传送给 Codec 芯片，经 D/A 转换得到模拟信号，再通过喇叭播放出来。

 版权所有 © 紫光展锐（上海）科技有限公司

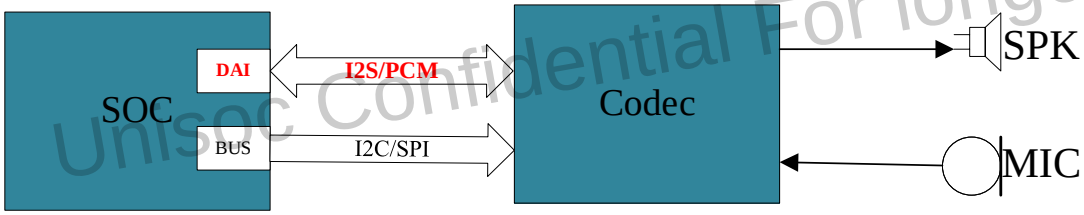
图1-5 音频的采集和播放



1.5 数字音频接口 DAI

数字音频接口 DAI 表示在板级内或板级间传输数字音频信号的方式。相比于模拟接口，数字音频接口抗干扰能力更强，硬件设计简单，目前常用数字音频接口有 I2S、PCM 和 AC97。

图1-6 数字音频接口 DAI



Unisoc Confidential For longcheer

在嵌入式平台，音频 Codec 芯片跟 CPU 的连接，一般分两个通道：

- 数据通道

  传送音频数据，如 I2S、PCM。
- 控制通道

  传送控制指令，如 I2C、SPI。

1.6 音频前后处理

音频处理是指对 PCM 数据进行处理，从而达到想要的效果，分为音频前处理和音频后处理。

- 音频前处理

  对音频编码前的 PCM 数据进行处理。主要用于语音中，去除各种干扰，使声音更清晰，包括回声消除（AEC）、噪声抑制（ANS）、自动增益控制（AGC）、静音抑制（VAD）等。

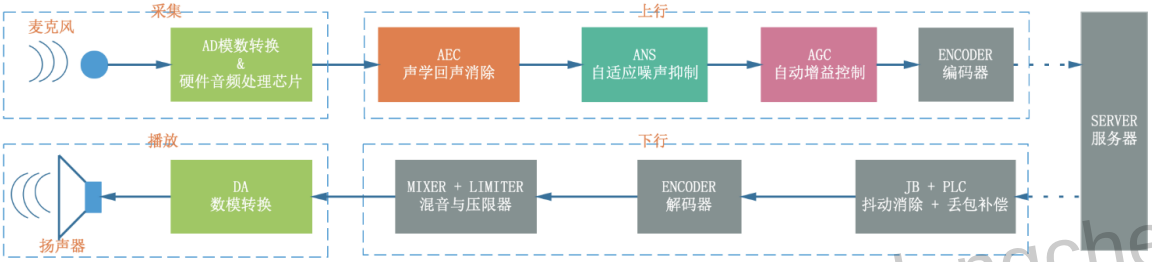
 版权所有 © 紫光展锐（上海）科技有限公司

- AEC（Acoustic Echo Canceler，回声消除器），可以消除各种延迟的回声。
- ANS（Automatic Noise Suppression，自动噪音抑制），可探测出背景固定频率的噪音（如风扇、空调声等）并自动滤除，呈现出说话人清晰的声音。
- AGC（Automatic Gain Control，自动增益控制），可以自动调整麦克风的收音量，使与会者收到一定的音量水平，不会因发言者与麦克风的距离改变时，声音有忽大忽小的缺点。
- VAD（Voice Activity Detection，语音活动检测），又称静音抑制。静音抑制的目的是从声音信号流里识别和消除长时间的静音期，以节省流量，它是 IP 电话应用的重要组成部分。在用户没有讲话时，就不会发送语音。在实际使用中，如果出现长时间的静默，会使用户感到很不自然。因此实际上接收端常常会在静音期间发送一些分组，从而生成使用户感觉舒服一些的背景噪声，即所谓的舒适噪声。

- 音频后处理

  对音频解码后的 PCM 数据进行处理。主要用于音乐中，产生各种音效，使音乐更动听，包括均衡器、混响等。

图1-7 音频前后处理



Unisoc Confidential For longcheer

1.7 音频编解码

如果把采样值直接保存或者发送，会占用很大的存储空间或者很大的流量。以 16kHz 采样率、16 位采样位数、单声道为例，一秒钟就有 16000*16*1/8=32000 字节（32KB）。通常需要把采样后的数字信号压缩后才保存或者发送。把采样值压缩叫编码（Encode），形成比特流。把比特流还原出采样值叫解码（Decode），编码和解码统称编解码（Codec）。

按压缩是否有损可分为无损压缩和有损压缩。

- 无损压缩

  把 PCM 数据压缩后无任何损伤，过压缩程度不高，如 FLAC/ALAC/APE/WMA Lossless 等。

- 有损压缩

  把 PCM 数据压缩后有损伤，最多可以压到几十分之一，这样的损伤往往是在可以接受范围之内，如 G711/G726/AAC/MP3/OGG 等。

1.8 语音通信中的 Tone

Tone 存在于日常打电话过程中。

 版权所有 © 紫光展锐（上海）科技有限公司

- 拨号音（Dial Tone）：使用固话拨号前，从听筒或扬声器听到的连续的“嗡”音，表示可以拨电话号码了。
- 回铃音（Ring Back Tone）：拨完号码对方振铃后，从听筒或扬声器听到的有规律断续的“嘟-嘟-”音，表示对方已振铃了。
- 忙音（Busy Tone）：对方占线时的提示音。
- 双音多频音（DTMF Tone）：给企业服务号（比如中国移动的 10086）打电话，对方提示按键选择，当按下按键后会听到的提示音。

在语音通信中 Tone 主要分两大类：CPT（Call Progress Tone，呼叫过程音）Tone 和 DTMF（Dual Tone Multi Frequency，双音多频音）Tone。

CPT Tone

CPT Tone 存在于通话过程中，主要用于告诉用户目前在什么状态，包括 Dial Tone/Ring Back Tone/Busy Tone 等。CPT Tone 是单频音，即由一个频率的正弦波形成。CPT Tone 没有全球统一的标准，而是各个国家有自己的标准，比如中国的标准、欧洲的标准、美国的标准等。

如**表 1-2** 所示，就是中国的 CPT Tone 标准。

表1-2 中国的 CPT Tone 标准

CPT Tone 名称	频率（Hz）	电平（dBm0）	播放形式
Dial Tone	450	-10	连续播放
Ring Back Tone	450	-10	放 1 秒停 4 秒，交替播放
Busy Tone	450	-10	放 350 毫秒停 350 毫秒，交替播放

Unisoc Confidential For longcheer

DTMF Tone

DTMF Tone 是双频音，即用两个频率（一个高频和一个低频）的正弦波叠加去表示某个按键值。DTMF Tone 全球有统一的标准。如**表 1-3** 所示，列出了 DTMF Tone 标准中常用的 16 个按键值是由哪些高频音和低频音组合而成的。

表1-3 DTMF Tone 标准中常用按键音的组成

低群	高群			
	1209Hz	1336Hz	1477Hz	1633Hz
697Hz	1	2	3	A
770Hz	4	5	6	B
852Hz	7	8	9	C
941Hz	*	0	#	D

 版权所有 © 紫光展锐（上海）科技有限公司

从软件实现的角度对 Tone 主要有两类处理：

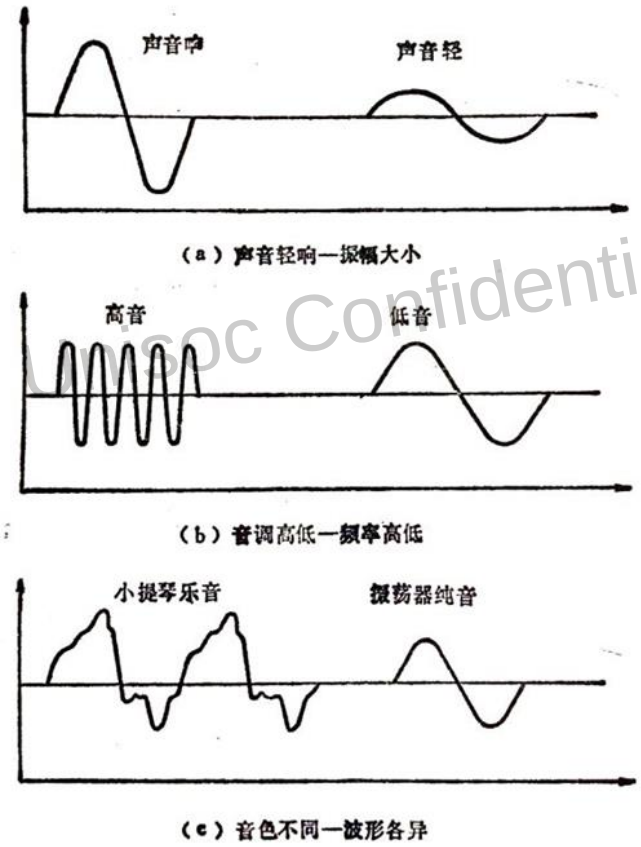
- Tone Generation，让用户听到 Tone。
- Tone Detection，主要是指 DTMF Tone 的 Detection，让软件知道哪个按键按下了，以便进行后续的处理。

1.9 响度/音调/音色

描述声音时，还会出现“响度、音调、音色”字眼，需要弄清他们代表的含义。

- 响度对应振幅大小。
- 音调对应频率高低。
- 音色是不同介质产生的差异。

图1-8 声音的性质和波形



Unisoc Confidential For longcheer

1.10 VoLTE/VoWiFi/VoIP

1.10.1 VoLTE

一直以来，市场上一直存在着两种语音传输的形式：

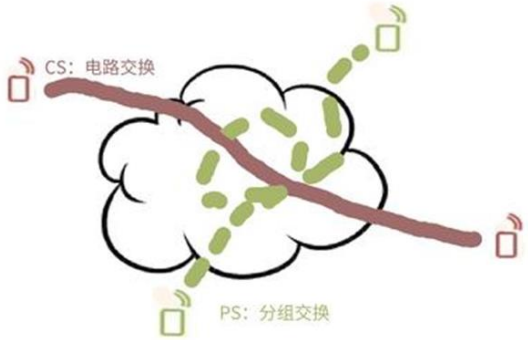
- CS（Circuited Switched，电路交换技术）

  所谓 CS，就是指通话前需在网络中建立一条线路，然后通过这条“独占”的线路传输语音资源，直到通话结束才拆除。

- PS（Packet Switched，分组交换技术）

  所谓 PS，则是数据打包传输，所有用户共有资源，然后通过事先指定路由的方式，将用户需要传输的语音资源送到指定的用户手上，无须建立“独占”线路。

图1-9 电路交换与分组交换



Unisoc Confidential For longcheer

在 2G、3G 时代，因为网速不足的缘故，运营商普遍采用传统的 CS 通讯方式，由于打电话时线路被独占，就无法上网。

进入 4G（LTE）时代后，数据网络网速大大提高，为了让大家共享 LTE 网络带宽，采用了 PS 技术进行语音传输，这就是 VoLTE。使用 VoLTE 可以边打电话边上网。

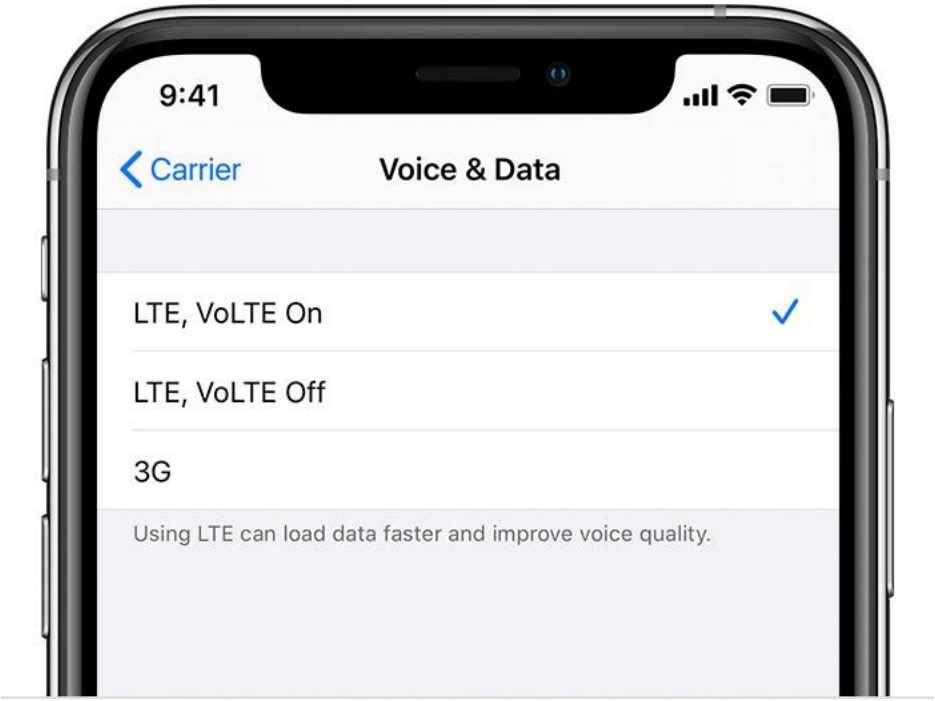
VoLTE 是 Voice over LTE 的缩写，就是利用 4G LTE 网路来承载语音封包来取代传统的语音通话。相比于 2G 和 3G，VoLTE 语音更为清晰可靠，支持 IP 视频通话，以及上网和语音同时业务。

要在 4G LTE 网络中打电话，有两个方案：

- 方案一：如果终端设备打开了 VoLTE，则默认使用 VoLTE，完全在 4G LTE 网络上传送语音。
- 方案二：当没有 4G LTE 信号时，将语音回落到 2G/3G，这叫 CSFB（CS Fall Back）。CSFB 最大的缺点是需依赖于 CS 核心网和 2G/3G 无线网络。

 版权所有 © 紫光展锐（上海）科技有限公司

图1-10 VoLTE



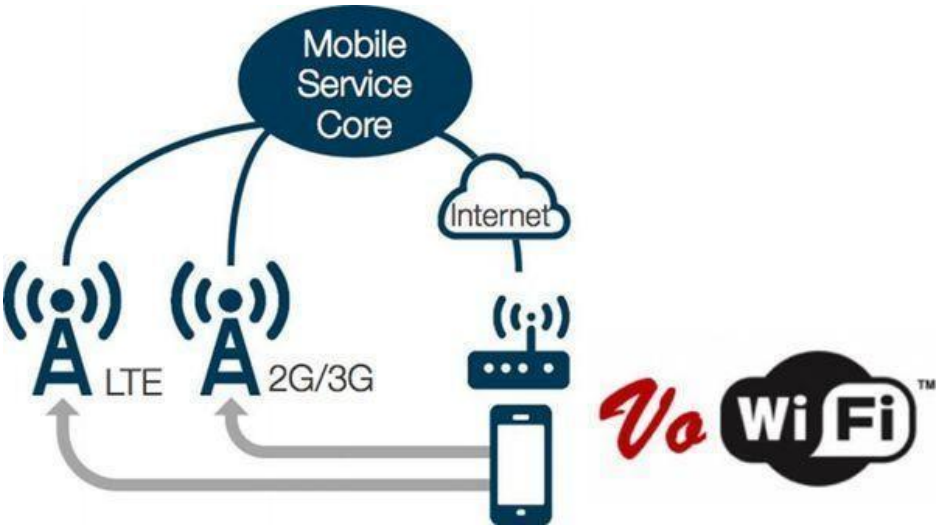
1.10.2 VoWiFi

Unisoc Confidential For longcheer

在 VoLTE 成熟经验基础上，把 LET 网络替换成 Wi-Fi 网络，即把 Wi-Fi 作为接入网，然后再接入运营商核心网实现打电话功能，这就是 VoWiFi。

VoWiFi 全名为 Voice over Wi-Fi，它和 VoLTE 相似，都是以数据的方式传递通话语音封包，但是中间传输的过程由 LTE 更换为 Wi-Fi 无线网路，把原来由基站完成的无线通信部分改为通过 Wi-Fi 完成，之后依然是进入运营商的核心网。

图1-11 VoWiFi



 版权所有 © 紫光展锐（上海）科技有限公司

VoWiFi 使用场景：运营商可以通过 Wi-Fi 解决室内覆盖问题，即在没有基站信号的情况下，用 Wi-Fi 替代基站实现通话。

由于 VoWiFi 最终也是要接入运营商核心网，跟 VoLTE 一样也要收费，收费方式是一样的。

目前在国内还很少看到 VoWiFi 的应用（VoWiFi 在国内难以落地），原因是：VoWiFi 在精确溯源方面没有满足相关政策要求。

1.10.3 VoIP

VoIP（Voice over IP）指的是在 Internet 上传送语音数据，比如微信语音。它跟 VoWiFi 不同，VoIP 并没有接入运营商核心网，所以可免费通话，只是消耗上网流量而已。

随着智能终端的普及，很多 VoIP 应用于移动终端设备，统称为 mVoIP（mobile VoIP）。可以通过 3G/4G 网络使用 mVoIP，也可以通过 Wi-Fi 使用 mVoIP，但是这和 VoWiFi 是有区别的。

1.10.4 VoLTE/VoWiFi/VoIP 差异比较

表1-4 VoLTE/VoWiFi/VoIP 差异比较

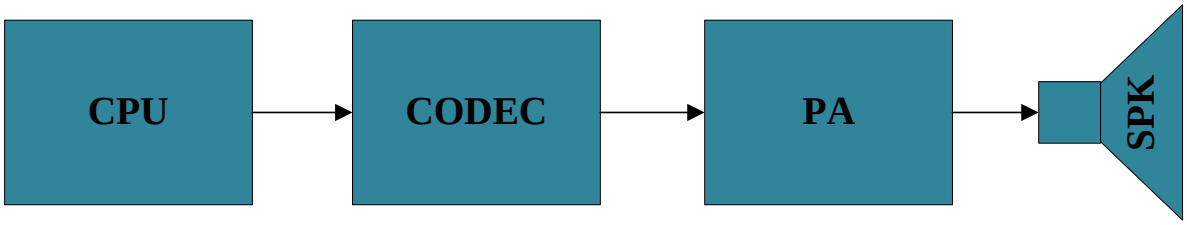
通话方式	工作模式	通话可靠性	优点	缺点
VoLTE	• 基于运营商 IMS，不需要安装 App • 接入基站，收费	高	稳定且通话质量高	室内或特殊地点存在网络不能连续性覆盖
VoWiFi	• 基于运营商 IMS，不需要安装 App • 接入 Wi-Fi，收费	中	低成本下扩展了语音业务	Wi-Fi 网络质量不可控，支持的用户数受限
VoIP	网络传送，免费（如微信语音通话）	低	免费	质量和稳定性无法保证，常见丢包和时延问题

Unisoc Confidential For longcheer

1.11 模拟功放/数字功放/Smart PA

Audio PA（Power Amplifier，功率放大器）在音频通路中的位置如图 1-12 所示。

图1-12 Audio PA 在音频通路中的位置



Audio PA 分为模拟 PA 和数字 PA，它们是从 PA 音频信号的输入接口来区分的：

 版权所有 © 紫光展锐（上海）科技有限公司

- 模拟 PA

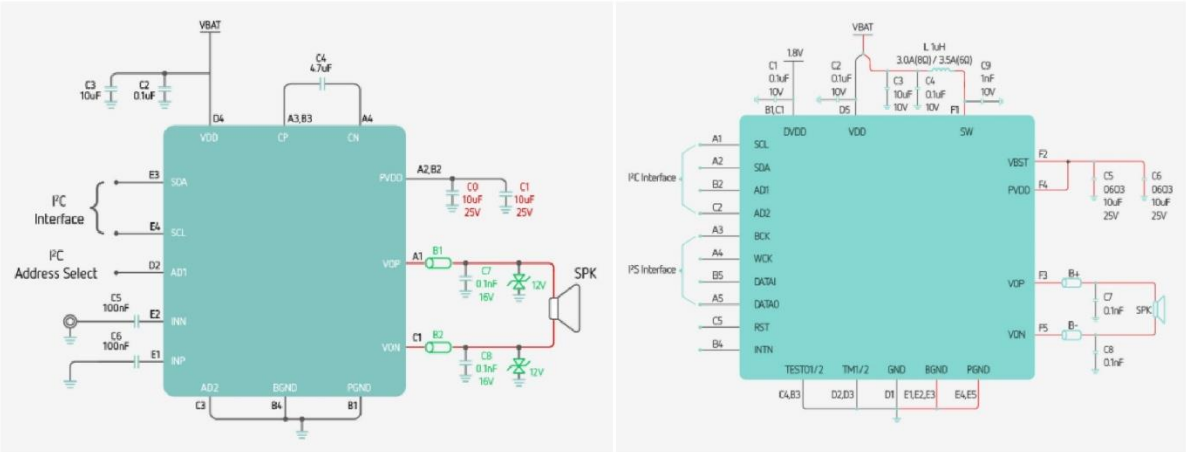
  音频信号从模拟接口输入。

- 数字 PA

  音频信号从 I2S 等数字接口输入。

如图 1-13 所示，左边是模拟 PA（音频信号是从双线 INP\INN 模拟接口输入）；右边是数字 PA（音频信号从 I2S 数字接口输入）。

图1-13 模拟 PA 和数字 PA



而 Smart PA 是带喇叭保护算法的 PA，自动调节输出频响和幅度，需要有算法模块支持。一般来说都会需要加一些控制逻辑在驱动里，所以基本上都是用数字 PA 的方式来实现的。

数字 PA 和 Smart PA 是两个概念，只不过大部分 Smart PA 都用数字 PA 的方式来实现而已。

Unisoc Confidential For longcheer

1.12 耳机和耳机座

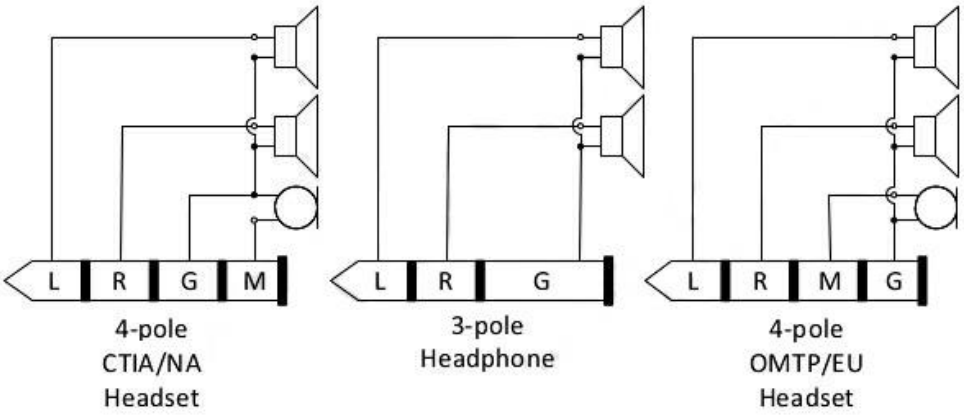
市面上流通的耳机从接口大小上分为 3.5mm 和 2.5mm 两种，主要适配不同尺寸的插口，比较常见的是 3.5mm。从接口电气特性上分为三段式和四段式（四段式在三段式耳机的基础上增加了 MIC 端）。

四段式耳机分美标（CTIA）和欧标（OMTP），国内用的是欧标。主要区别在于耳机线侧最后两端的定义：

- 美标：L（左声道）、R（右声道）、G（GND）、M（MIC）。
- 欧标：L、R、M、G。

 版权所有 © 紫光展锐（上海）科技有限公司

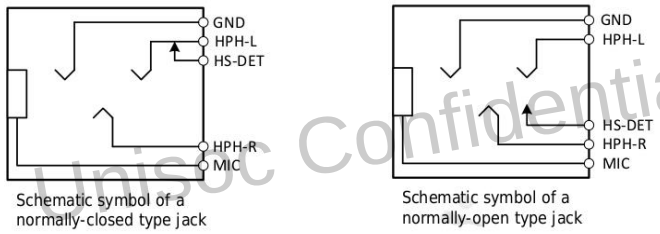
图1-14 耳机接口



相应的，耳机座（Jack）也分为支持欧标设计的耳机座和支持美标设计的耳机座。

另外，从耳机座左声道的检测方式来又可以分为“normally-closed type”（常闭型，简称 NC）和“normally-open type”（常开型，简称 NO）两种，展锐硬件参考设计使用的是“常开型耳机座”。

图1-15 耳机座



Unisoc Confidential For longcheer

- 常闭型耳机座

  不接耳机时，耳机座左声道和检测端 HS-DET 导通；插入耳机时，HS-DET 与 HPH-L 断开。
- 常开型耳机座

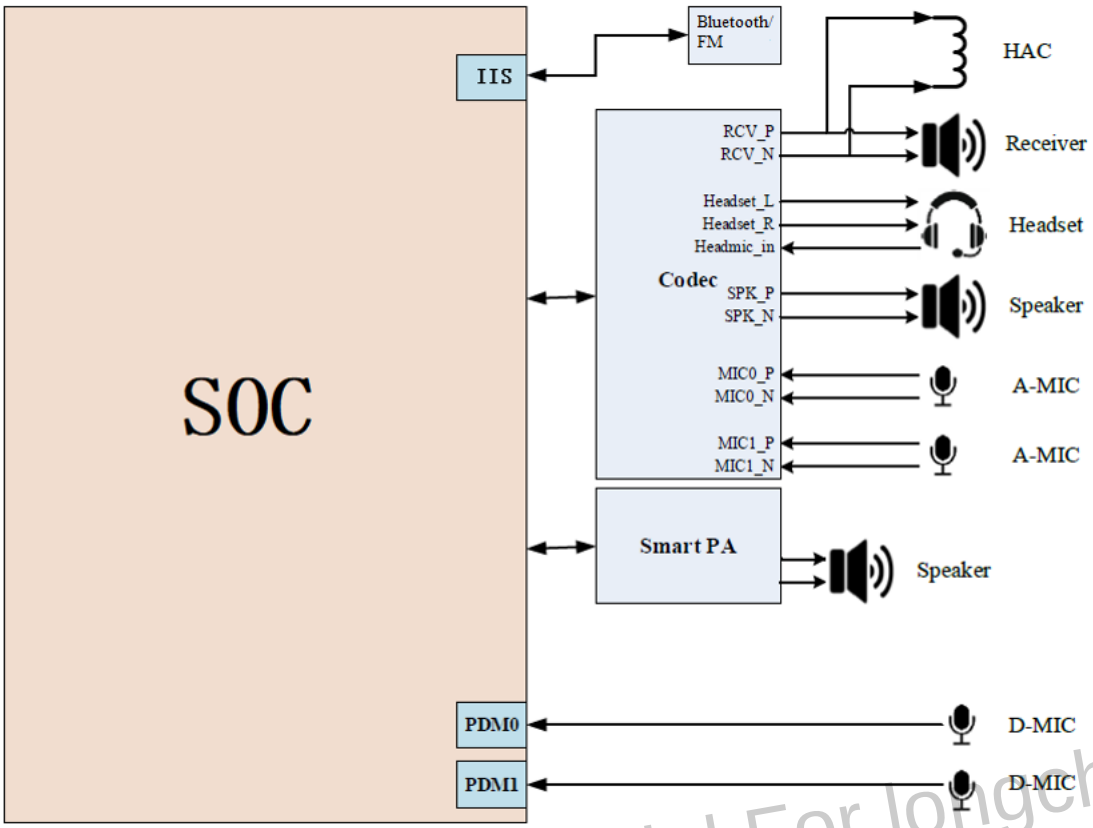
  不接耳机时，耳机座左声道和检测端 HS-DET 断开，插入耳机时，HS-DET 与 HPH-L 导通。

1.13 音频设备

如图 1-16 所示，图中画出了常见的各种音频设备（包括音频输入设备和音频输出设备）。

 版权所有 © 紫光展锐（上海）科技有限公司

图1-16 各种各样的音频设备



Unisoc Confidential For longcheer

- **Bluetooth®**/FM：蓝牙/调频收音机
- HAC：助听器
- Receiver：听筒
- Headset：耳机
- Speaker：扬声器
- A-MIC：模拟麦克风
- D-MIC：数字麦克风

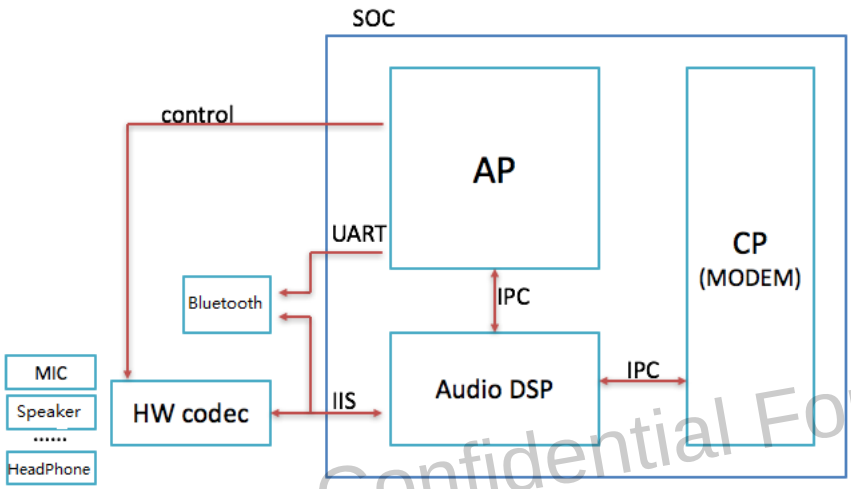
 版权所有 © 紫光展锐（上海）科技有限公司

2 Android 设备音频架构

2.1 硬件架构

如图 2-1 所示，为目前主流 Android 智能终端设备的音频硬件架构。

图2-1 终端设备音频硬件架构



Unisoc Confidential For longcheer

- AP（Application Processor，应用处理器），现在使用最多的是 Arm® Processor，运行的是 Android 操作系统和应用程序。
- CP（Communication Processor，通信处理器），主要处理跟通信相关的程序。
- Audio DSP 是专门用于处理音频的处理器。功耗是终端设备的一个重要指标，也是通过认证的必选项。有了 Audio DSP，在打电话和听音乐的应用场景下，可以让 AP 在绝大多数时间内都处于 Sleep 状态，这样可以有效降低终端设备的功耗。
- AP、CP 和 Audio DSP 之间通过 IPC（Inter-Processor Communication）通信，交互控制消息和音频数据。通常 AP、CP 和 Audio DSP 集成在一个 SOC 芯片内。
- 用于音频采集和播放的硬件 Codec 芯片，它受 AP 控制（使能以及选择不同音频路径等，主要是配置寄存器），与 Audio DSP 通过 I2S 总线交换音频数据。
- 连接硬件 Codec 的有各种外设，如 MIC、Earpiece（听筒）、Speaker（扬声器）、有线耳机等。
- 蓝牙设备
  - 当使用蓝牙耳机听音乐时，音频码流在 AP 上解码成 PCM 数据，使用 A2DP 协议经过 UART 直接传送给蓝牙芯片（而不是经过 Audio DSP，通过 I2S 总线传送给蓝牙芯片）。
  - 蓝牙通话时，CP 和 Audio DSP 之间通过 IPC 交互语音数据，Audio DSP 跟蓝牙芯片之间通过 I2S 接口交互语音数据。

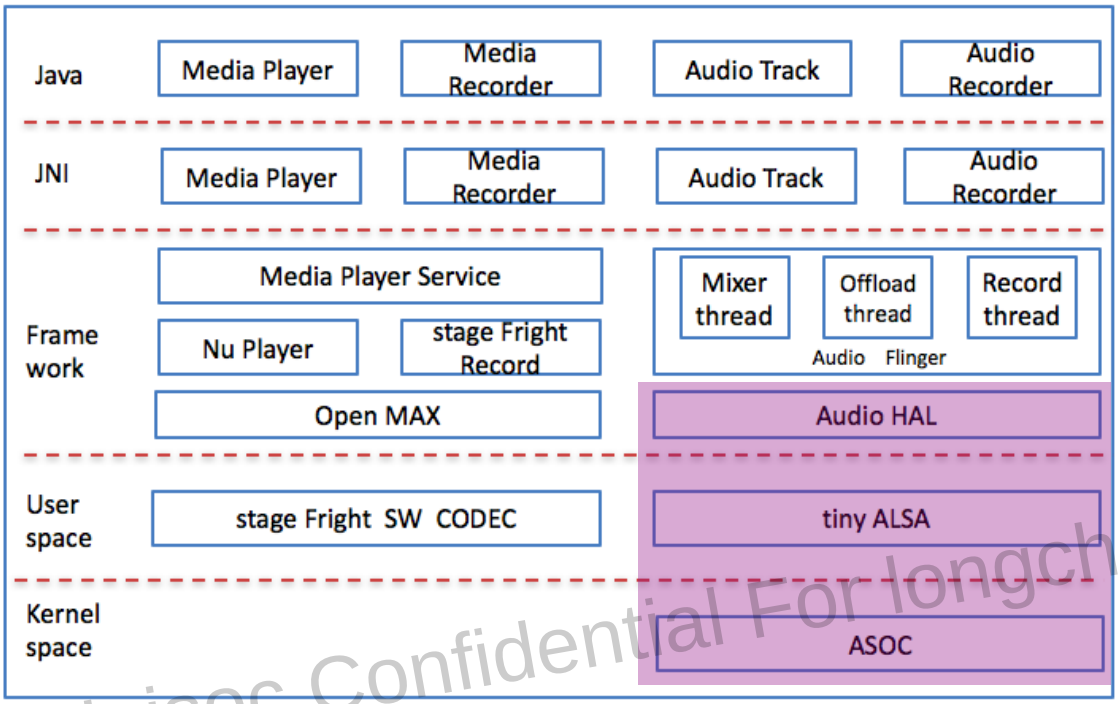
 版权所有 © 紫光展锐（上海）科技有限公司

2.2 软件架构

2.2.1 AP 软件架构

AP 侧运行的是 Android 系统，其 Audio 部分的软件架构如图 2-2 所示。

图2-2 AP 侧 Audio 软件架构



Unisoc Confidential For longcheer

Android 音频软件分不同的层，主要有 Kernel/User/HAL（Hardware Abstract Layer，硬件抽象层）/Framework/JNI/Java。

- Kernel space 和 User space 层：Android 的底层核心是 Linux，Linux 上音频子系统是 ASOC（ALSA System on Chip），包括 Kernel space 和 User space 两部分。Kernel space 里是音频驱动，User space 里主要是提供统一的 API 给应用程序调用。Android 在 User space 里对 ALSA 进行裁剪形成了 tiny ALSA（图 2-2 中右下角紫色部分）。

 版权所有 © 紫光展锐（上海）科技有限公司

图2-3 ALSA 子系统架构

```
              +--------+  +--------+  +--------+
              |tinyplay|  |tinycap |  |tinymix |
              +--------+  +--------+  +--------+
                   |           ^           ^
                   V           |           V
              +-----------------------------------+
              |        ALSA Library API           |
              |      (tinyalsa, alsa-lib)         |
              +-----------------------------------+
  user space                  ^
-------------------------------|---------------------
  kernel space                V
              +-----------------------------------+
              |           ALSA CORE               |
              | +-------+ +-------+ +------+      |
              | |  PCM  | |CONTROL| | MIDI |...|
              | +-------+ +-------+ +------+      |
              +-----------------------------------+
                                  |
              +-----------------------------------+
              |           ASoC CORE               |
              +-----------------------------------+
                                  |
              +-----------------------------------+
              |        hardware driver            |
              |  +--------+ +-------+ +-----+     |
              |  |Platform| |Machine| |Codec|     |
              |  +-------+ +--------+ +-----+     |
              +-----------------------------------+
                     |                   |
                     V                   V
              数据流(如I2S)        控制流（如I2C）
```

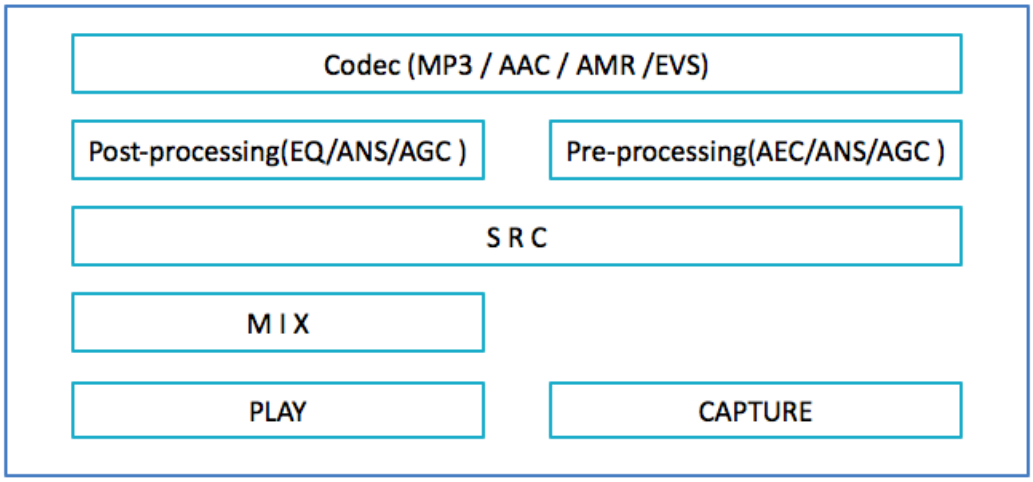
Unisoc Confidential For longcheer

- Audio HAL：音频硬件抽象层，对硬件设备的抽象和封装，向下调用 Linux 系统 tiny ALSA 接口，向上为 FW 层提供统一的访问硬件设备的接口，为上层屏蔽了不同硬件设备的差异。Audio HAL 在 Android 中隶属于 Framework 层。
- Framework 层：里面细分多层，模块众多，也是很复杂的一层。FW 上层是 JNI，提供接口供 Java 调用，提供的接口主要有 Media Recorder/Media Player/Audio Track/Audio Recorder 等。
- App 层：最上层的 Java 层，也就是各种带有音频功能的 App（比如录音机、录像机等）。

 版权所有 © 紫光展锐（上海）科技有限公司

2.2.2 DSP 软件架构

图2-4 Audio DSP 软件架构



Audio DSP 模块主要有 Codec（MP3/AAC/AMR/EVS 等）、前处理（AEC/ANS/AGC 等）、后处理（EQ/ANS/AGC 等）、重采样（SRC）、混音（MIX）、从 DMA 获取采集到的音频数据（CAPTURE）、将音频数据送给 DMA 后播放（PLAY）等。当然还有接收和发送给其他处理器的音频数据处理等，AP 和 CP 都要与 Audio DSP 交互语音数据。

Unisoc Confidential For longcheer

2.2.3 不同场景音频数据流向

Android 终端设备中音频的主要场景有音频播放、音频录制、语音通信等。不同场景下的音频数据流向有很大差异，即使是同一场景，在不同的模式下音频数据流向也有所不同。

2.2.4 音频播放

播放模式

Android 目前支持 3 种播放模式：Deep Buffer Playback、Low Latency Playback 和 Compressed Offload Playback。

- Deep Buffer Playback

  Android 开发中最常用的播放模式，用于音乐等对时延要求不高的声音输出。边加载边播放，音频文件是在 AP 侧解码成 PCM 数据，再送入 Audio Flinger 进行混音及音效处理，最后经由 Audio DSP 送给 Codec 芯片播放出来。

- Low Latency Playback

  低延迟播放模式，用于按键音、游戏背景音等对时延要求高的场景。一次加载完数据，应用一般采用 Soundpool 播放，也是在 AP 侧解码成 PCM 数据，经由 Audio DSP 送给 Codec 芯片播放出来。

- Compressed Offload Playback

  也是用于音乐播放，但是音频解码部分的工作是在 Audio DSP 中完成，即俗称的硬解码。AP 侧只负责把音频码流送到 Audio DSP 中，送出去后 AP 侧会进行休眠。Audio DSP 中会分配一块较大的 Buffer 去处理此数据，在 Audio DSP 中进行解码、音效的处理等工作。在 Audio DSP 解码器处理完

 版权所有 © 紫光展锐（上海）科技有限公司

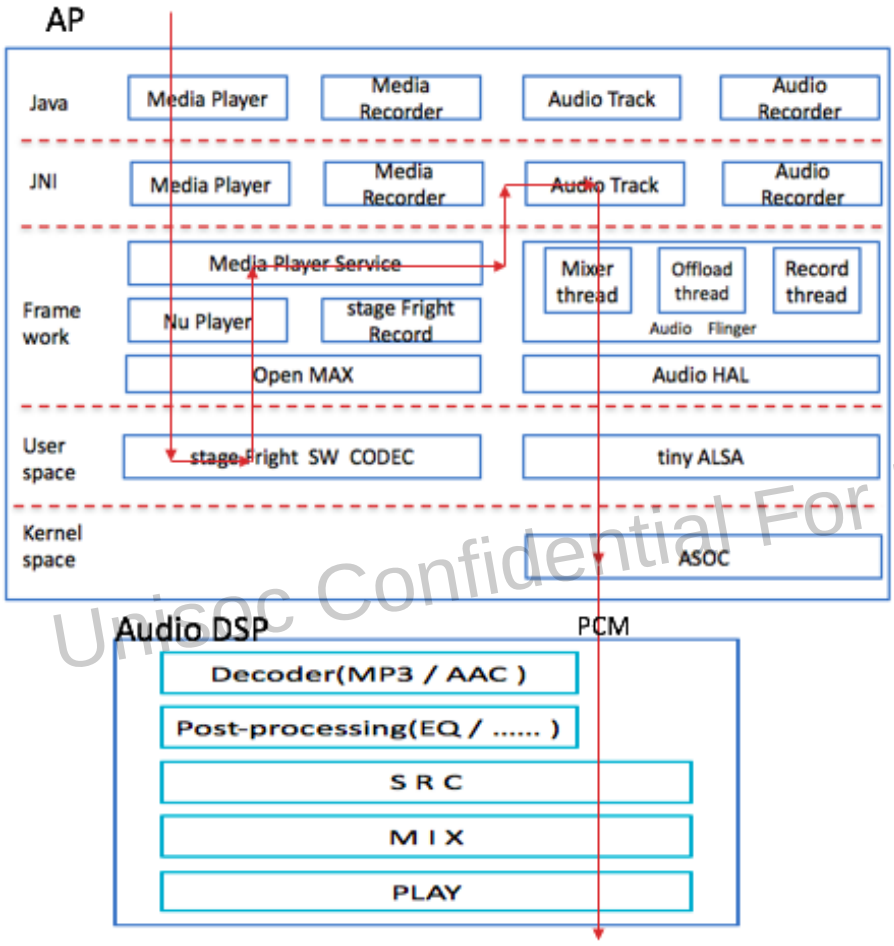
数据之前，它会唤醒 AP 去送下一包数据。用这种模式播放音频能有效的降低功耗，目前展锐平台只支持 MP3 用 Offload 模式播放。

综上，Low Latency 模式和 Deep Buffer 模式都是在 AP 侧解码完后送 PCM 数据给 Audio DSP，称之为软解码；而 Compressed Offload 模式是码流送给 Audio DSP 解码，称之为硬解码。

音频播放软解码

音频播放软解码：Low Latency/Deep Buffer 模式下的音频数据流向，如图 2-5 所示。

图2-5 音频播放软解码



Unisoc Confidential For longcheer

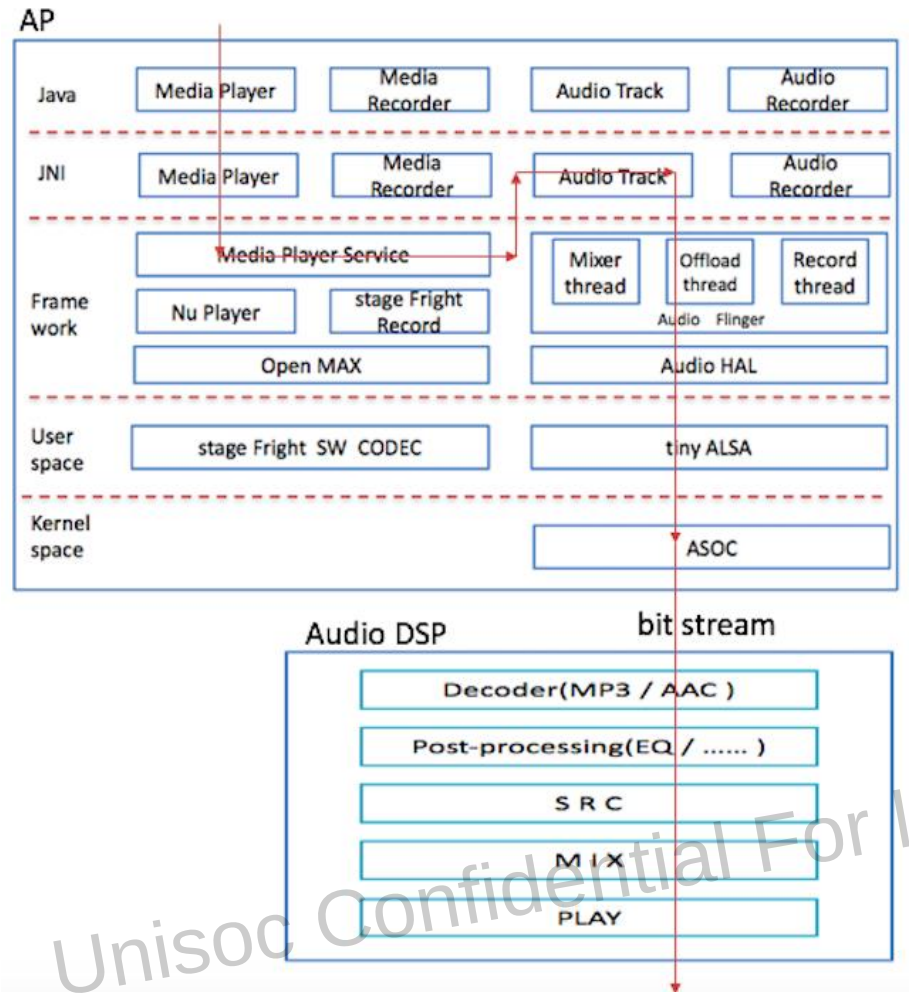
音频文件先在 AP 上软解码，得到 PCM 数据后经过 Audio Track/Audio Flinger 中的 Mixer thread（有可能要做音效后处理）/Audio HAL/tiny ALSA 后送给 Kernel，然后通过 IPC 将 PCM 数据送给 Audio DSP，经重采样、混音等后再播放出来。由于在 AP 上已做解码和音效后处理，Audio DSP 上就不需要做解码了。

音频播放硬解码

音频播放硬解码：compressed offload 模式下的音频数据流向，如图 2-6 所示。

 版权所有 © 紫光展锐（上海）科技有限公司

图2-6 音频播放硬解码



Unisoc Confidential For longcheer

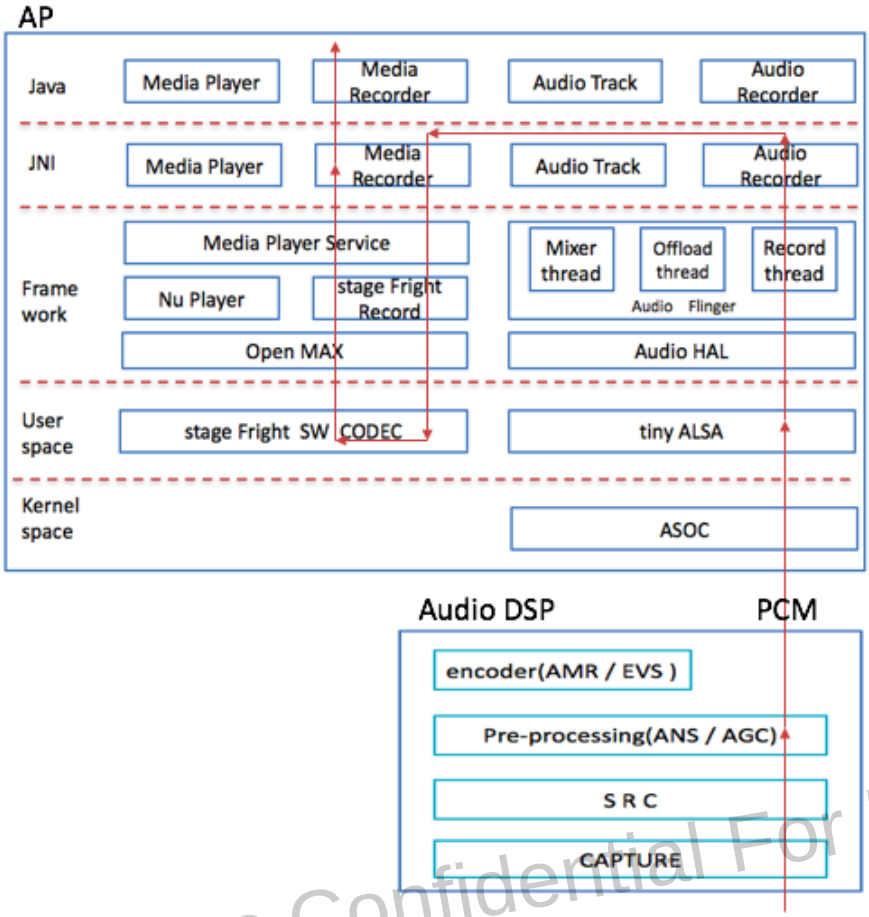
音频码流经过 AP 上的 Audio Track/Audio Flinger 中的 Offload thread（不需要做音效后处理）/Audio HAL/tiny ALSA 后送给 Kernel，然后通过 IPC 将码流送给 Audio DSP，经解码、后处理、重采样、混音等再播放出来。

2.2.5 音频录制

如图 2-7 所示，为录音时音频数据的流向。同音频播放一样，录音时音频数据也是只经过 AP 和 Audio DSP。

 版权所有 © 紫光展锐（上海）科技有限公司

图2-7 音频录制



Unisoc Confidential For longcheer

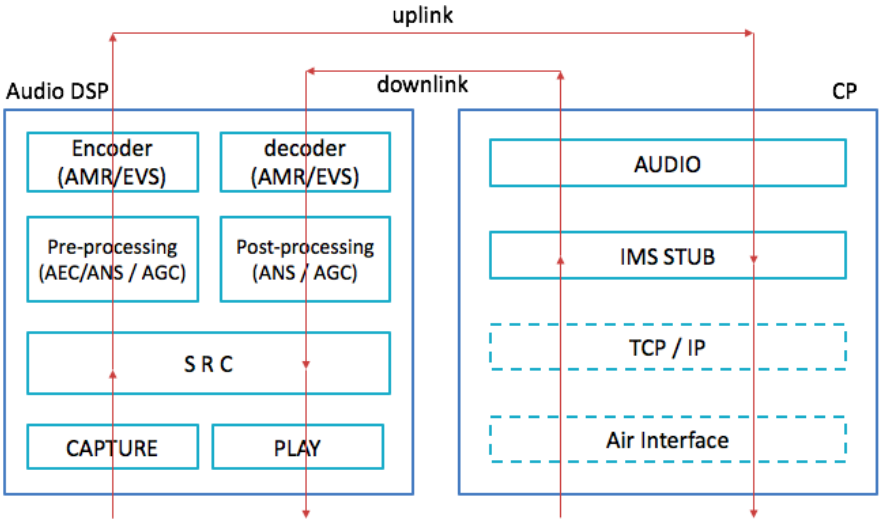
Codec 芯片采集到的 PCM 数据送给 Audio DSP 经重采样、前处理后送给 AP 的 Kernel，再经由 tiny ALSA/Audio HAL/Audio Flinger 中的 Record thread/Audio Record 等后，做软编码得到录音码流文件并保存起来。

2.2.6 语音通话

语音通话（即打电话），同音频播放或者录制不一样，它是双向的，分上行（Uplink，把采集到的语音发送给对方）和下行（Downlink，把收到的语音播放出来），而音频播放或者录制则是单向的。它的音频数据流向也跟音频播放或者录制不一样，只经过 Audio DSP 和 CP。如图 2-8 所示，就是打电话时音频数据的流向。

 版权所有 © 紫光展锐（上海）科技有限公司

图2-8 语音通话



- 在上行方向上，Codec 芯片采集到的 PCM 数据送给 Audio DSP 经重采样、前处理（AEC/ANS/AGC 等）、编码后得到码流，并将码流送给 CP，CP 处理后经过空口（Air Interface）发送给对方。
- 在下行方向上，先从空口收对方送过来的语音数据，并做网络侧处理，然后送给 Audio DSP，Audio DSP 收到后做解码、后处理（ANS/AGC 等）、重采样等，再把 PCM 数据经 DMA/I2S 送给 Codec 芯片播放出来。

Unisoc Confidential For longcheer

3 常用调试方法

3.1 YLog 和 Logcat

提供日志最为常用的调试方法，反馈问题时务必都带上日志。如何使用 YLog，可在 Uni-Support 系统上搜索对应 Android 版本的 YLog 使用指南文档，如《Android 10.0 YLog 抓取使用指南》。

3.2 Dump Audio HAL

声音有卡顿、破音、回音、杂音、噪音等异常时，可以通过以下命令 Dump 出底层音频进行分析。该方法只能 Dump 到 AP 侧的声音。以下情况通过该方式是 Dump 不到的：

- 连接蓝牙播放音乐和蓝牙通话（但电话振铃的声音可以 Dump 到）
- 打电话的过程（但拨号按键音可以 Dump 到）

另外，仅 Userdebug 版本可以 Dump。

Unisoc Confidential For longcheer

Android 8.1

```
adb root
adb shell
echo set_dump_data_switch=0x0fff > /data/local/media/mmi.audio.ctrl
echo set_dump_data_path=/data/local/media/ > /data/local/media/mmi.audio.ctrl
```

Android 9/10/11

```
adb root
adb shell
echo set_dump_data_switch=0x0fff > /data/vendor/local/media/mmi.audio.ctrl
echo set_dump_data_path=/data/vendor/local/media/ > /data/vendor/local/media/mmi.audio.ctrl
```

将/data/local/media/或/data/vendor/local/media/目录下新增的 dump_*开头的文件 Pull 出来，在电脑上播放看数据是否有异常。

3.3 Dump Audio DSP

DSP Log 和 Modem Log，得用 Logel 工具抓取和分析。抓取有两种方法：使用 YLog 离线抓取，或者使用 Logel 在线实时抓取。

 版权所有 © 紫光展锐（上海）科技有限公司

通话和 VoIP 音质类问题都需要抓取 Audio Log 来进行分析，YLog 场景要先设置为语音场景。详细的操作可参考 Uni-Support 系统上的《UNISOC 音频 LOG 抓取和分析方法指导》文档。

3.4 tinyalsa 调试命令

tinyalsa 配套工具有 tinyplay、tinymix、tinycap、tinypcminfo、tinyhostless 等。Android 编译默认是没有编译这些工具的，需要手动局部编译：

```
cd external/tinyalsa/
mm
```

- tinymix：查看和设置混音控制参数

```
# tinymix -h
tinymix [options] [control name/#] [value to set]
    options:
    --device|-D <card#>      - use the given card # instead of 0.
    --all-values|-a          - show all possible values/ranges for control.
    --tabs-only|-t           - separate all output columns/values with tabs.
    --value-only|-v          - show only the value for the selected control.
```

- tinyplay：播放音频

```
# tinyplay
Usage: tinyplay file.wav [-D card] [-d device] [-p period_size] [-n n_periods]
```

- tinycap：录音

```
# tinycap
Usage: tinycap file.wav [-D card] [-d device] [-c channels] [-r rate] [-b bits] [-p period_size] [-n n_periods] [-T capture time]
```

- tinypcminfo：查看 pcm 通道的相关信息

```
# tinypcminfo
Usage: ./tinypcminfo -D card -d device
```

- tinyhostless：录播回环

```
# tinyhostless
Usage: ./tinyhostless [-D card] [-P playback device] [-C capture device] [-p period_size] [-n n_periods] [-c num_channels] [-r sample_rate] [-l] [-T playback/capture time]

Used to enable 'hostless' mode for audio devices with a DSP back-end.
Alternatively, specify '-l' for loopback mode: this program will read
from the capture device and write to the playback device.
```

说明

tinyplay 和 tinycap 前先要打开对应音频通路，否则会报错。由于不同平台打开音频通路不同，需要用到时可提交 CQ 进行咨询。

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

3.5 查看声卡与设备

查看系统已注册声卡

```
# cat /proc/asound/cards
 0 [sprdphone          ]: sprdphone - sprdphone
                          sprdphone
 1 [alli2s             ]: all-i2s - all-i2s
                          all-i2s
 2 [saudiovoip         ]: saudiovoip - saudiovoip
                          saudiovoip
```

查看系统已注册设备

```
# cat /proc/asound/devices
  0: [ 0]      : control
 16: [ 0- 0]: digital audio playback
 17: [ 0- 1]: digital audio playback
 20: [ 0- 4]: digital audio playback
 21: [ 0- 5]: digital audio playback
 24: [ 0- 0]: digital audio capture
 25: [ 0- 1]: digital audio capture
 26: [ 0- 2]: digital audio capture
 27: [ 0- 3]: digital audio capture
 32: [ 1]      : control
 33:              : timer
 48: [ 1- 0]: digital audio playback
 56: [ 1- 0]: digital audio capture
 64: [ 2]      : control
 80: [ 2- 0]: digital audio playback
 81: [ 2- 1]: digital audio playback
 88: [ 2- 0]: digital audio capture
```

只查看 PCM 设备列表

```
# cat /proc/asound/pcm
00-00: vbc-r1p0v3-sprd-codec-normal sprd-codec-normal-0 :    : playback 1 : capture 1
00-01: vaudio-sprd-codec-voice sprd-codec-voice-1 :    : playback 1 : capture 1
00-02: vbc-r1p0v3-ad23-sprd-codec-ad1 sprd-codec-ad1-2 :    : capture 1
00-03: vaudio-ad23-sprd-codec-ad1-voice sprd-codec-ad1-voice-3 :    : capture 1
```

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

```
00-04: vbc-dfm-sprd-codec-fm sprd-codec-fm-4 :    : playback 1
00-05: vbc-deep-buf-sprd-codec-offload-playback sprd-codec-offload-pla :    : playback 1
01-00: i2s_bt_sco0-snd-soc-dummy-dai snd-soc-dummy-dai-0 :    : playback 1 : capture 1
02-00: SAUDIO PCM : SAUDIO PCM : playback 1 : capture 1
02-01: SAUDIO PCM : SAUDIO PCM : playback 1
```

查看声卡里面的 Codec 信息（不同平台文件节点会有差异）

```
# cat /proc/asound/card0/sprd-codec
sc27xx-audio-codec digital part
0x0000 | 0x0000 0x0000 0x0780 0x03f2
0x0010 | 0x0000 0x0008 0x0000 0x0120
0x0020 | 0x0000 0x0000 0x0000 0x0003
0x0030 | 0x0000 0x0018 0x9999 0x0001
sc27xx-audio-codec analog part(0x403C8700)
0x0000 | 0x0000 0x0000 0x0000 0x0000
0x0010 | 0x0000 0x0000 0x0000 0x0000
0x0020 | 0x0000 0x0000 0x0000 0x0000
0x0030 | 0x0000 0x0000 0x0000 0x0000
0x0040 | 0x0000 0x0000 0x0000 0x0000
0x0050 | 0x0000 0x0000 0x0000 0x0000
0x0060 | 0x0000 0x0000 0x0000 0x0000
0x0070 | 0x0000 0x0000 0x0000 0x0000
sc27xx-audio-codec analog part(0x403C8800)
0x0000 | 0x0000 0x0000 0x0000 0x0000
0x0010 | 0x0000 0x0000 0x0000 0x0000
0x0020 | 0x0000 0x0000 0x0000 0x0000
0x0030 | 0x0000 0x0000 0x0000 0x0000
sc27xx-audio-codec audif(0x403C8140)
0x0000 | 0x0000 0x0000 0x0000 0x0000
0x0010 | 0x0000 0x0000 0x0000 0x0000
```

Unisoc Confidential For longcheer

3.6 查看 GPIO 状态

```
# cat /d/gpio
GPIOs 0-255, platform/40280000.gpio-controller, sprd-ap-gpio:
 gpio-32   (                    |6 Key                 ) in   hi
 gpio-50   (                    |?                     ) out lo
```

```
gpio-51   (                    |GPIO_TE          ) in   lo
gpio-85   (                    |8 Key            ) in   hi
gpio-88   (                    |7 Key            ) in   hi
gpio-90   (                    |9 Key            ) in   hi
gpio-91   (                    |Up Key           ) in   hi
gpio-92   (                    |5 Key            ) in   hi
gpio-138 (                     |GPIO_CD          ) out lo
gpio-144 (                     |4 Key            ) in   hi
gpio-146 (                     |Menu Key         ) in   hi
gpio-147 (                     |Right Key        ) in   hi
gpio-154 (                     |0 Key            ) in   hi
gpio-155 (                     |Star Key         ) in   hi
gpio-156 (                     |Hash Key         ) in   hi
```

3.7 lookat 操作寄存器

比如操作 Codec 部分的寄存器（各个平台 Codec 寄存器地址不同）：

```
# lookat -l 4 0x40388700                    // 开启录音机之前
[I] nword = 0x4
   ADDRESS   |    VALUE
-----------+-----------
0x40388700 | 0x00000000
0x40388704 | 0x00000000
0x40388708 | 0x00000000
0x4038870c | 0x00000000
# lookat -l 4 0x40388700                    // 开启录音机之后
[I] nword = 0x4
   ADDRESS   |    VALUE
-----------+-----------
0x40388700 | 0x0000bc4b
0x40388704 | 0x0000832a
0x40388708 | 0x000012a6
0x4038870c | 0x0000e429
# lookat -s 0x0000b44b 0x40388700       // bit11 置 0，Audio BIAS EN 0 = disable，此刻录音为噪音
[I] value = 0xb44b
# lookat -s 0x0000bc4b 0x40388700       // bit11 置 1，Audio BIAS EN 1 = enable，录音恢复正常
```

Unisoc Confidential For longcheer

```
[I] value = 0xbc4b
```

Unisoc Confidential For longcheer

4 常见问题汇总

4.1 声卡注册失败

这类问题往往都是由于修改了驱动或者硬件客制化带来的资源冲突所造成的。比如：

例 1

【问题描述】

修改音频驱动改出 Bug，比如设备树中 GPIO（General Purpose Input Output，通用输入/输出接口）定义冲突，Codec DAI 定义有误等。以下两个 GPIO 定义冲突，导致内核空指针调用，声卡注册失败，设备重启。

```
&sprd_headset {
        status = "okay";
        /*  这是有关 Type-C 模拟耳机的配置  */
        extcon = <&pmic_typec>;
        mic-gpios = <&ap_gpio 16 GPIO_ACTIVE_HIGH>;
        lr-gpios = <&ap_gpio 15 GPIO_ACTIVE_HIGH>;
        sprd,3pole-adc-threshold = <112>;
};

&dsi {
        status = "okay";
        #address-cells = <1>;
        #size-cells = <0>;

        panel: panel {
                compatible = "sprd,generic-mipi-panel";
                #address-cells = <1>;
                #size-cells = <0>;
                reg = <0>;

                avdd-gpio = <&ap_gpio 46 GPIO_ACTIVE_HIGH>;
                avee-gpio = <&ap_gpio 15 GPIO_ACTIVE_HIGH>;
```

Unisoc Confidential For longcheer

```
            reset-gpio = <&ap_gpio 50 GPIO_ACTIVE_HIGH>;
            port {
                reg = <1>;
                panel_in: endpoint {
                    remote-endpoint = <&dphy_out>;
                };
            };
        };
};
```

【解决方案】

在展锐开发板上，设备树中 GPIO15 是用于 Type-C 模拟耳机的。当没用到 Type-C 模拟耳机时，通过删掉有关 Type-C 耳机的配置，来解决跟 GPIO15 的冲突。

```
&sprd_headset {
            status = "okay";
-            extcon = <&pmic_typec>;
-            mic-gpios = <&ap_gpio 16 GPIO_ACTIVE_HIGH>;
-            lr-gpios = <&ap_gpio 15 GPIO_ACTIVE_HIGH>;
-            sprd,3pole-adc-threshold = <112>;
};
```

Unisoc Confidential For longcheer

例 2

【问题描述】

硬件差异引起，没有用到展锐推荐的 Smart PA（型号 UCP1301），导致声卡注册失败。

【解决办法】

屏蔽掉有关 UCP1301 的配置。

 版权所有 © 紫光展锐（上海）科技有限公司

```
diff --git a/old/bsp/kernel/kernel4.14/arch/arm64/boot/dts/sprd/ums512-1h10.dts b/new/bsp/kernel/kernel4.14/arch/arm64/boot/dts/sprd/ums512-1h10.dts
index 1236b4f..5817485 100755
--- a/old/bsp/kernel/kernel4.14/arch/arm64/boot/dts/sprd/ums512-1h10.dts
+++ b/new/bsp/kernel/kernel4.14/arch/arm64/boot/dts/sprd/ums512-1h10.dts
@@ -526,12 +526,14 @@
        status = "okay";
        clock-frequency = <400000>;

+       /*
        ucp1301_spk: ucp1301@58 {
                compatible = "sprd,ucp1301-spk";
                reg = <0x58>;
                reset-gpios = <&ap_gpio 134 GPIO_ACTIVE_HIGH>;
                #sound-dai-cells = <1>;
        };
+       */

        fan54015_chg: charger@6a {
                compatible = "fairchild,fan54015_chg";
@@ -808,13 +810,14 @@
                "MIC2 Pin", "Aux Mic Jack",
                "inter HP PA", "HP Pin",
                "inter Spk PA", "SPK Pin",
-               "UCP1301 SPK ON", "SPK Pin",
-               "inter Spk PA", "UCP1301 SPK",
+               //"UCP1301 SPK ON", "SPK Pin",
+               //"inter Spk PA", "UCP1301 SPK",
                "DMIC Pin", "DMic Jack",
                "DMIC1 Pin", "DMic1 Jack",
                "inter Ear PA", "EAR Pin";

        /* 35: ucp1301 normal ap01 */
+       /*
        sprd-audio-card,dai-link@53 {
                ignore-suspend = <1>;
                link-name = "BE_NORMAL_AP01_UCPSPK";
@@ -833,6 +836,7 @@
                        sound-dai = <&ucp1301_spk 0>;
                };
        };
+       */

        /*
         * node 54 and node 55 (index 36 and 37) are reserved for
diff --git a/old/bsp/kernel/kernel4.14/sprd-diffconfig/androidq/sharkl5Pro/arm64/ums512_1h10_diff_config b/new/bsp/kernel/kernel4.14/sprd-diffconfig/androidq/sharkl5Pro/arm64/ums512_1h10_diff_config
index 42054c1..b7b8081 100755
--- a/old/bsp/kernel/kernel4.14/sprd-diffconfig/androidq/sharkl5Pro/arm64/ums512_1h10_diff_config
+++ b/new/bsp/kernel/kernel4.14/sprd-diffconfig/androidq/sharkl5Pro/arm64/ums512_1h10_diff_config
@@ -1 +1,2 @@
 ADD:CONFIG_SPRD_MINI_SYSDUMP
+DEL:CONFIG_SND_SOC_SPRD_PA_UCP1301
```

例 3

【问题描述】

硬件差异引起，没有用到展锐推荐的 Type-C Switch（型号 FSA4476，用于 Type-C 模拟耳机），导致声卡注册失败。

【解决办法】

屏蔽对应配置。

Unisoc Confidential For longcheer

```
/* ums512-1h10.dts */
&sprd_headset {
        status = "okay";
-       extcon = <&pmic_typec>;
-       mic-gpios = <&ap_gpio 16 GPIO_ACTIVE_HIGH>;
-       lr-gpios = <&ap_gpio 15 GPIO_ACTIVE_HIGH>;
-       sprd,3pole-adc-threshold = <112>;
};
```

4.2 主辅 MIC 录音

默认情况下，录音机使用的是主 MIC 录音，摄像机使用的是辅 MIC。如果不想使用默认的，比如录音机要使用辅 MIC，摄像机要使用主 MIC，有两种方法进行修改：

- 从 App 层修改

  FW 层通过以下两个 AudioSource 来标识主辅 MIC。

  ```
  MediaRecorder.AudioSource.MIC          //主 mic
  ```

 版权所有 © 紫光展锐（上海）科技有限公司

```
MediaRecorder.AudioSource.CAMCORDER    //辅 MIC
```

在使用 mediarecord 或 audiorecord 接口时，设置对应 AudioSource 即可，比如摄像机改用主 MIC 录音：

```
-                    mMediaRecorder.setAudioSource(MediaRecorder.AudioSource.CAMCORDER);
+                    mMediaRecorder.setAudioSource(MediaRecorder.AudioSource.MIC);
```

- 从配置文件修改

如果是第三方 App，无法修改 App 层代码，可以从系统层面解决该问题，修改 primary_audio_policy_configuration.xml 文件，删掉有关辅 MIC 的配置，即系统不支持辅 MIC，FW 层发现配置不支持辅 MIC，代码会自动切换为主 MIC。

```
--- a/primary_audio_policy_configuration_old.xml
+++ b/primary_audio_policy_configuration_new.xml
@@ -5,7 +5,6 @@
            <item>Speaker</item>
            <item>Earpiece</item>
            <item>Built-In Mic</item>
-           <item>Built-In Back Mic</item>
        </attachedDevices>
        <defaultOutputDevice>Speaker</defaultOutputDevice>
        <mixPorts>
@@ -72,11 +71,7 @@
                        samplingRates="8000,11025,12000,16000,22050,24000,32000,44100,48000"
channelMasks="AUDIO_CHANNEL_IN_MONO,AUDIO_CHANNEL_IN_STEREO,AUDIO_CHANNEL_IN_FRONT_BACK"/>
            </devicePort>
-           <devicePort tagName="Built-In Back Mic" type="AUDIO_DEVICE_IN_BACK_MIC" role="source">
-               <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
-                       samplingRates="8000,11025,12000,16000,22050,24000,32000,44100,48000"
-
channelMasks="AUDIO_CHANNEL_IN_MONO,AUDIO_CHANNEL_IN_STEREO,AUDIO_CHANNEL_IN_FRONT_BACK"/>
-           </devicePort>
            <devicePort tagName="Wired Headset Mic" type="AUDIO_DEVICE_IN_WIRED_HEADSET" role="source">
                <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
                        samplingRates="8000,11025,12000,16000,22050,24000,32000,44100,48000"
@@ -108,9 +103,9 @@
            <route type="mix" sink="BT SCO Car Kit"
                    sources="primary output,Telephony Rx"/>
```

Unisoc Confidential For longcheer

```
        <route type="mix" sink="primary input"
-                sources="Built-In Mic,Built-In Back Mic,Wired Headset Mic,BT SCO Headset Mic"/>
+                 sources="Built-In Mic,Wired Headset Mic,BT SCO Headset Mic"/>
        <route type="mix" sink="Telephony Tx"
-                sources="Built-In Mic,Built-In Back Mic,Wired Headset Mic,BT SCO Headset Mic"/>
+                 sources="Built-In Mic,Wired Headset Mic,BT SCO Headset Mic"/>
        <route type="mix" sink="voice_rx"
                 sources="Telephony Rx"/>
    </routes>
```

4.3 双 MIC 录音

双 MIC 录音，往往应用于使用第三方音频处理算法的场景，如回音消除、双 MIC 降噪等。

双 MIC 降噪方案的原理：硬件上将辅 MIC 的信号直接接在 SPK 输出上，辅 MIC 的输入信号将会记录到录音音源的右声道，左声道则记录主 MIC 的录音数据。双 MIC 录音分别存在左右声道上并传到 App，App 层再传给第三方算法处理模块，用于做回声抑制算法处理。

Android 系统的录音默认都是单 MIC 录音，展锐基线版本默认是不支持双 MIC 录音的，只能提交 CQ 进行客制化修改。

Unisoc Confidential For longcheer

4.4 外置模拟 PA

这是音频客制化的高频场景之一。手机产品内置喇叭比较固定，通常不会改变。但工业电子不同行业产品多，扬声器的接法则是多种多样，比如：

- 通过 PMIC 的 SPK 信号接扬声器（手机默认方式）
- 通过 PMIC 的 EAR 信号外接扬声器
- 通过 PMIC 的耳机左声道外接单扬声器
- 通过 PMIC 的耳机右声道外接单扬声器
- 通过 PMIC 的耳机左右声道外接双扬声器
- 通过 Smart PA 外接扬声器

模拟 PA 的形式也是多样，控制接口有 I2C、SPI 之分，对应驱动也不同。为了做到低功耗，驱动要实现有声音的时候才开启 PA，没声音时要自动关闭 PA。也有些应用完全不控制 PA，设备一开机就一直供电，此时驱动处理也有不同。

外置模拟 PA 要改动的地方包括：

- Uboot：外部 PA 工作状态通常使用 GPIO/I2C 来控制，GPIO 的引脚属性需正确配置，需要修改 Uboot 中的 Pinmap 文件。
- Kernel：GPIO、I2C、Audio 通路等设备树均要修改。
- HAL：路由通路配置文件需要根据情况进行修改。

 版权所有 © 紫光展锐（上海）科技有限公司

这部分的修改比较多，有专门的参考文档，不同芯片、不同 Android 版本的文档有所不同，具体可以咨询展锐的技术支持人员。

4.5 外置数字 PA 及第三方 Codec

通过 I2S 外接 Smart PA 或者外接第三方 Codec 实现多变的音频使用场景，是客制化最为高频的场景之一，比如：

- 外接 SLIC 话机
- 外接蓝牙模块（非展锐自带的蓝牙）
- 外接 FM 模块
- 外接 Codec 实现多路 MIC 采集（天猫精灵语音识别等）

这部分代码改动会比较多，联调也比较麻烦，难度也会比较大。I2S 时序和参数相对比较复杂，展锐芯片和第三方 Codec 的 I2S 参数需要一致，若有不匹配会出现声音异常等问题。

另外，各个 SOC 的驱动有差异，驱动移植方法也不一样。Uni-Support 系统上有一些指导性的文档，不同平台文档不同，具体可以咨询展锐的技术支持人员。

修改代码比较多，无法逐一展示，总体思路是：

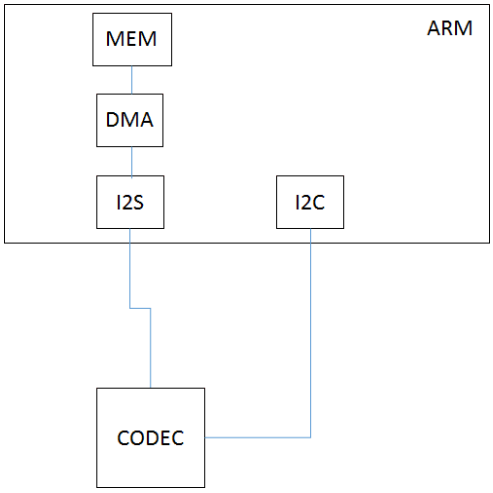
1. 先修改驱动，保证 I2S 声卡驱动能注册成功。
2. 使用 tinycap 和 tinyplay 验证 I2S 声卡工作是否正常，看能否录音和播放。如果还没接 MIC 和喇叭的，可以先用信号分析仪进行分析。
3. 根据使用场景调整音频通路。
4. 调整 NV 参数，以适配具体的硬件（喇叭和 MIC）。

Unisoc Confidential For longcheer

播放流程

硬件上数字音频总线接口有很多（如 I2S、PCM、TDM 等），以 I2S 为例对播放流程进行说明（录制过程相反）：

图4-1 I2S 接口接第三方 Codec



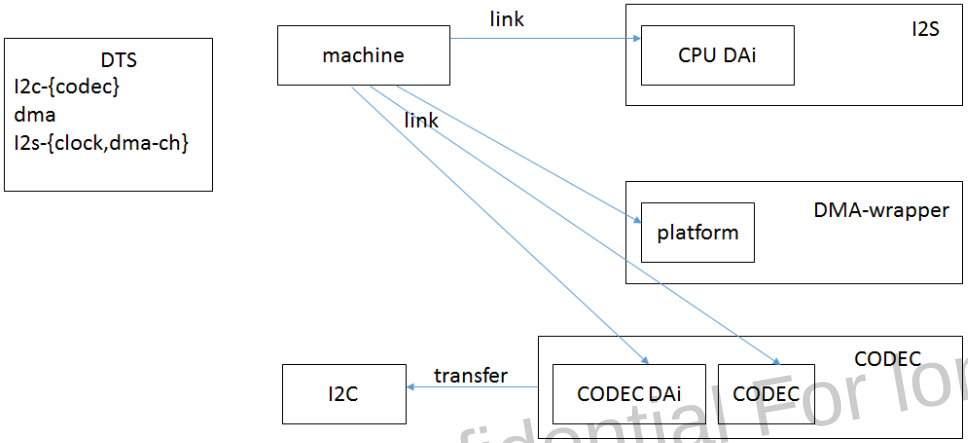
 版权所有 © 紫光展锐（上海）科技有限公司

1. 使用 I2C 对 CODEC 做初始化配置。
2. 音频数据从外部 Flash 或网络（VoIP）读取到内存中。
3. 数据经过 DMA 送到 I2S 的发送缓冲区 TXFIFO。
4. 数据由 TXFIFO 送到 Codec。
5. 在 Codec 里，数据经过 D/A 从数字信号转变成模拟信号。
6. 模拟信号经过功放和喇叭播放出来。

驱动程序

按照 ALSA 驱动框架要实现的接口，如图 4-2 所示。

图4-2 按照 ALSA 驱动框架要实现的接口



Unisoc Confidential For longcheer

各个部分功能分别是：

- DTS（Device Tree Source File，设备树源文件）：音频设备硬件参数定义。
- CPU DAI：CPU 上的数字音频接口驱动，如 I2S、PCM、TDM 等。
- Platform：基于 CPU 的 DMA 接口封装，用于实现音频数据在内存和 DMA 之间传送。
- Codec DAI：Codec 上的音频接口驱动。
- CODEC：Codec 自身驱动。
- I2C：实现 Codec 的配置接口。
- Machine：设备层面组装所有驱动，实现音频通路。

4.6 耳机和耳机座

展锐参考设计的硬件和软件默认都是按“常开式耳机座”来设计的，如果耳机不匹配或者硬件设计不匹配就会带来一些问题。

- 声音异常

  比如使用欧标耳机声音异常。
- 耳机检测问题

 版权所有 © 紫光展锐（上海）科技有限公司

  - 耳机检测不到
  - 耳机类型检测错误
  - 检测门限电压修改
  - HEADMIC Bias 使能关闭（消除噪音）
- 耳机 Button 问题
  - 耳机 Button 无效
  - 耳机按键功能不对
  - 耳机 Button 误触发

如果要使用“常闭式耳机座”，这是客制化需求，请提 CQ 给展锐。

4.7 其它音频问题

音频问题比较复杂，需要具体问题具体分析，以下仅列举部分情况。

4.7.1 回音

打电话或者 VoIP 通话的时候，有回音消除，这是在 DSP 内部处理的。回音消除功能没起作用，可能是硬件上提供的回音参考信号设计有误，或者 NV 参数配置有误。

4.7.2 噪音

I2S 参数配置不对，或硬件设计不符合规范，或 NV 参数未适配硬件，都有可能产生噪音。

4.7.3 Pop 音

在通过耳机音频通路接外置 PA 的情况下，经常会出现 Pop 音（在开始播放和停止播放时，都能明显听到一个 Pop 音。这个 Pop 音是由 Codec 内部充放电产生的。

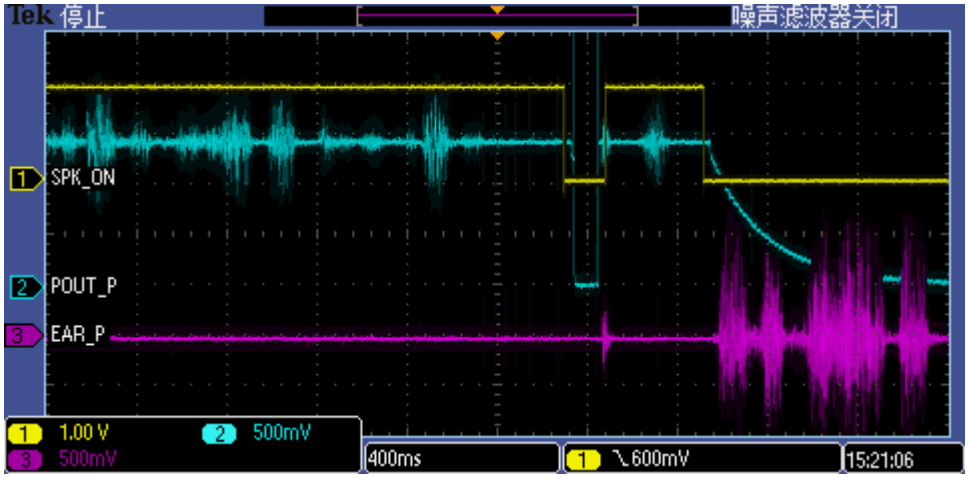
一般消除 Pop 音是通过控制 PA 上电时序来实现的，开关 Codec 时要保证外部 PA 处于关闭状态。

如图 4-3 所示，为使用示波器抓取的 PA 使能脚的信号波形。对应场景为：PA 使能脚关掉一段时间后又打开一段时间，最后再关掉。

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

图4-3 PA 使能脚关掉的波形示例



对于“既要 PA 使能脚常开（不能关闭 PA），又要没 Pop 音”的需求，可以让 Codec 对应的音频通路一直处于上电状态（缺点是牺牲了功耗）。

4.7.4 没声音

应用代码、通路配置、驱动、硬件、NV 参数等有错误都有可能引起没声音的问题，有时候还只是上行（或下行）没声音。列举如下：

- App 应用音频焦点混乱导致无音。
- NV 参数配置错误导致蓝牙通话下行有声，上行无声。
- 硬件 WCN 模块有问题导致蓝牙没声音。
- 其他，音频通路配置有误、SoC 和外置 Codec 的 I2S 参数不匹配、音频驱动等问题都可能导致没声音，只能具体问题具体分析。

Unisoc Confidential For longcheer

4.7.5 音量小

由于产品喇叭音腔结构等设计的不同，当所有可调的音量参数都调到最大，还感觉声音不够大时，就需要调整 NV 参数，提高 Codec 内部的音频模拟增益。具体可提交 CQ，联系展锐技术支持人员进行音频参数调试。

 版权所有 © 紫光展锐（上海）科技有限公司